NAME: AKSHAYA A

REG NO:192421222

COURSE: CSA0603 DESIGN AND ANALYSIS FOR ALGORITHMS

CO4_AT2_INDUSTRY PROBLEM SOLVING TASK

Travelling Salesman Problem (TSP) for Logistics Route Optimization Using Dynamic Programming – Held-Karp Algorithm

Introduction

A logistics company needs to deliver packages to **15 cities**. A delivery vehicle starts from a central depot, visits every city **exactly once**, and finally returns to the depot.

The objective is to find a route with the **minimum total travel cost or distance**.

For a small number of cities, the Travelling Salesman Problem can be solved exactly using **Dynamic Programming**, particularly the **Held-Karp algorithm**.

Real-Time Example: Courier Delivery Network

Consider a courier company operating from a **Chennai distribution depot**.

The company has one delivery vehicle that needs to serve 15 locations during a planned delivery cycle. The locations may include Bengaluru, Hyderabad, Coimbatore, Madurai, Trichy, Salem, Kochi, Mysuru, Vijayawada, Tirunelveli, Pondicherry, Vellore, Thanjavur, Erode and Visakhapatnam.

The vehicle must follow this pattern:

Chennai Depot → Visit all 15 cities exactly once → Chennai Depot

The company estimates the travel cost between every pair of locations based on factors such as distance, fuel consumption and toll charges.

The challenge is to determine the **least-cost route**.

The cities and route structure in this example are illustrative of a logistics scenario.

Problem Statement

The objective is to find the minimum-cost tour where:

- The vehicle starts from the depot.
- Every city is visited exactly once.
- No city is visited more than once.
- The vehicle returns to the depot.
- The total travelling cost is minimized.

The problem becomes increasingly difficult as the number of cities increases because the number of possible routes grows rapidly.

Why TSP Is Important in Logistics

TSP-based optimization can help logistics companies reduce:

- Total travelling distance
- Fuel consumption
- Transportation cost
- Delivery time
- Unnecessary travelling
- Operational expenses

An optimized route can help a delivery vehicle complete its planned deliveries using fewer resources.

Held-Karp Dynamic Programming Approach

The **Held-Karp algorithm** solves TSP using Dynamic Programming.

Instead of calculating every complete route independently, it stores solutions to smaller subproblems and reuses them.

The main idea is:

Find the minimum cost of reaching a particular city after visiting a particular set of cities.

State Representation

The Dynamic Programming state is represented as:

[
DP[S][j]
]

where:

- **S** = set of cities already visited
- **j** = current city
- **DP[S][j]** = minimum cost of starting from the depot, visiting every city in S, and finishing at city j.

For example:

[
DP[{1,2,4}][4]
]

means:

The minimum cost of starting from the depot, visiting cities 1, 2 and 4, and ending at city 4.

Bitmask Representation

A bitmask can be used to represent the visited cities.

For example:

Cities: 1 2 3 4

Visited: 1 0 1 0

This means that cities **1 and 3** have already been visited.

For 15 cities:

[
2^{15}=32768
]

possible subsets exist.

This makes subset-based Dynamic Programming practical for the given problem size.

Recurrence Relation

The main Held-Karp recurrence is:

[
DP[S][j]

\min_{i\in S,\ i\neq j}
\left(
DP[S-{j}][i] + C_{ij}
\right)
]

where:

- S represents the current set of cities.
- j represents the current city.
- i represents the previous city.
- Cij represents the travelling cost from city i to city j.

In simple terms:

To reach city j, find the cheapest previous city i and add the cost of travelling from i to j.

Initial Condition

The initial state is:

[
DP[{j}][j] = C_{0j}
]

Here, 0 represents the starting depot.

This means the initial cost of visiting city j is the cost of travelling directly from the depot to city j.

Final Step

After all cities have been visited, the vehicle must return to the depot.

Therefore:

[
Answer =
\min_j
\left(
DP[All][j] + C_{j0}
\right)
]

The algorithm selects the final city that gives the lowest total cost after returning to the depot.

Working of the Algorithm

The complete process is:

Start

↓

Create distance/cost matrix

↓

Represent cities using bitmask

↓

Initialize DP states

↓

Generate subsets of cities

↓

Calculate minimum cost for each state

↓

Store results in DP table

↓

Visit all cities

↓

Return to depot

↓

Find minimum total cost

↓

Reconstruct optimal route

↓

End

Example of Route Decision

Suppose the algorithm considers routes such as:

Depot → A → B → C

Depot → B → A → C

Depot → C → A → B

Instead of recalculating the same partial routes repeatedly, the Dynamic Programming table stores the best result for each subset.

For example:

[
DP[{A,B}][B]
]

stores the cheapest way to visit A and B and finish at B.

If another route requires the same state, the stored value is reused.

This reduces repeated calculations.

Complexity Analysis

The Held-Karp algorithm has:

Time Complexity

[
O(n^2 2^n)
]

Space Complexity

[
O(n2^n)
]

For 15 cities:

[
2^{15}=32768
]

possible subsets exist.

Therefore, a 15-city problem is considerably more manageable than a much larger TSP instance.

Scalability Issues

The biggest limitation of Held-Karp is its **exponential growth**.

For example:

[
2^{20}=1,048,576
]

and:

[
2^{30}=1,073,741,824
]

As the number of cities increases, the number of DP states increases extremely rapidly.

This can result in:

- High memory usage
- Longer computation time
- Large DP tables
- Poor scalability
- Difficulty handling real-time route changes

Therefore, Held-Karp is mainly suitable for relatively small TSP problems.

Feasibility in Real-World Logistics

For a logistics problem involving **15 cities**, Held-Karp is a suitable exact approach because:

- The number of locations is relatively small.
- An optimal route can be obtained.
- The algorithm provides an exact solution.
- The result can be used for route planning.
- It provides a useful baseline for evaluating other optimization methods.

However, actual logistics systems are more complicated.

They may involve:

- Multiple vehicles
- Vehicle capacity restrictions
- Delivery time windows
- Different delivery priorities
- Driver availability
- Traffic conditions
- Fuel limitations
- Road restrictions

When these constraints are included, the problem becomes closer to the **Vehicle Routing Problem (VRP)** rather than the basic TSP.

Tools for Implementation

The solution can be implemented using:

Python

Python is suitable because it provides simple data structures for implementing Dynamic Programming and bitmasking.

VS Code

VS Code can be used to develop, test and debug the program.

NumPy

NumPy can be used for numerical calculations and handling distance matrices.

Matplotlib

Matplotlib can be used to visualize cities and the final optimized route.

Jupyter Notebook

Jupyter Notebook can be used to experiment with different numbers of cities and compare execution times.

Real road-distance data can also be obtained from mapping or geographic data sources when building a practical logistics prototype.

Performance Limitations

The major limitation is the exponential complexity:

[
O(n^2 2^n)
]

For 15 cities, this can be practical.

As the number of cities increases, however, the number of possible states grows rapidly.

For example:

[
n=15 \Rightarrow 2^{15}=32768
]

while:

[
n=30 \Rightarrow 2^{30}=1,073,741,824
]

Therefore, using Held-Karp directly for hundreds or thousands of delivery locations is not practical.

Improvements for Large Logistics Networks

For larger problems, companies can use alternative optimization approaches such as:

- Nearest Neighbor
- Greedy algorithms
- Genetic Algorithms
- Simulated Annealing
- Ant Colony Optimization
- Branch and Bound
- Vehicle Routing algorithms
- Specialized optimization solvers

These methods may not always provide the exact mathematical optimum, but they can produce good routes much faster for large-scale logistics problems.

Result

For the illustrative 15-city courier network, the Held-Karp algorithm evaluates

The selected route is the one with the **minimum total travelling cost**.

Advantages

1. Produces an exact optimal solution for the standard TSP.
2. Avoids repeated calculations using Dynamic Programming.
3. Bitmasking efficiently represents visited cities.
4. Suitable for small logistics networks.
5. Provides a strong baseline for comparing heuristic methods.
6. Gives a mathematically justified minimum-cost route.

Limitations

1. Exponential time complexity.
2. High memory requirement for larger inputs.
3. Not suitable for very large numbers of cities.
4. Basic TSP does not consider vehicle capacity.
5. Multiple vehicles require a more advanced model.
6. Dynamic traffic conditions require additional optimization.
7. Real-time route changes may require recalculation.

Conclusion

The **Held-Karp Dynamic Programming algorithm** provides an effective exact solution for a logistics company that needs to visit **15 cities exactly once and return to the depot**.

The state DP[S][j] represents the minimum cost of visiting a subset of cities and ending at city j. The recurrence builds larger solutions from previously calculated smaller solutions.

For the 15-city case, the approach is feasible and can produce an exact optimal route. However, its **(O(n^2 2^n))** complexity creates serious scalability limitations as the number of cities increases.

Therefore, Held-Karp is suitable for **small logistics optimization problems**, while large real-world delivery networks generally require heuristics, approximation techniques, or Vehicle Routing Problem algorithms.